오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 심화

DAY 20

비동기 심화와 브라우저 저장소

요청의 실패를 나눠 다루고 로딩 · 오류 · 성공을 상태로 그린 뒤, localStorage로 새로고침 뒤에도 데이터를 남깁니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **요청의 실패를 나눠 다룰 수 있다**

   ok · 상태 코드 · 인터넷 연결 · 시간 초과를 구분해, 사람이 읽을 안내 문구로 바꿉니다.

2. **로딩 · 오류 · 성공을 상태로 나눌 수 있다**

   화면에 보일 것을 상태 객체 하나에 두고, render 함수 하나로 모든 모습을 그립니다.

3. **localStorage · sessionStorage로 데이터를 남길 수 있다**

   배열을 JSON 글자로 바꿔 저장하고, 새로고침 뒤에 다시 읽어 화면을 되살립니다.

오늘의 결과물

상태를 나눠 그리는 댓글 보기 comments.js와, 새로고침해도 남는 할 일 목록 todo.js

ROADMAP

오늘의 흐름

01

fetch 응답 심화

응답 들여다보기
실패마다 다른 안내

02

async 구조화

역할별 세 함수
차례로 · 함께 기다리기

03

상태 분리

로딩 · 오류 · 성공
상태와 render · 실습 01

실습 포함

04

브라우저 저장소

local · sessionStorage
JSON 저장 · 실습 02

실습 포함

앞의 두 섹션은 DAY 18의 fetch를 실패까지 나눠 다루고, 요청 코드를 역할별 함수로 나눕니다.
뒤의 두 섹션은 화면을 상태로 그리는 법과, 새로고침 뒤에도 남는 브라우저 저장소입니다.

되짚기 · 01

ok부터 확인한 fetch

배운 곳	그때 본 것	오늘
DAY 18 28 · 29장	response.ok — 404여도 fetch는 이행	요청 함수 하나에 ok 확인
DAY 18 30장	ok가 아니면 throw — catch로	오류에 상태 코드를 담아 던지기
DAY 18 38 · 40장	실패는 세 곳 — catch 하나로 받음	실패마다 다른 안내 문구
DAY 18 37장	finally — 성공해도 실패해도 마지막에	상태 하나로 버튼까지

오늘은 실패를 나눠서 다룹니다

무엇이 실패했는지에 따라 다른 문구를 보이고, 다시 시도할 길을 줍니다.

되짚기 · 02

이미 써 본 async 함수와 render

배운 곳	그때 본 것	오늘
DAY 17 40장	async 함수는 언제나 Promise를 돌려줌	역할별로 함수 나누기
DAY 17 36장 · DAY 18 41장	Promise.all — 여러 개를 함께 기다림	차례로와 함께 견주기
DAY 18 44장	데이터를 바꾸고, 화면은 render로 다시	상태 객체 + render 하나
DAY 16 실습 02	Todo List — 새로고침하면 사라짐	localStorage로 남기기

DAY 16의 할 일 목록을 새로고침해도 남게 만드는 날

그 전에, 요청 코드를 상태로 나눠 정리합니다.

준비

오늘의 셋팅

```
Day20/                          ← 같은 저장소 안에 만듭니다
  index.html                    ← 미리 받은 파일 (장표 예제)
  index.js                      ← 장표를 보며 함께 칩니다
  style.css                     ← 모두 함께 씁니다
  comments.html · comments.js   ← 실습 01
  todo.html · todo.js           ← 실습 02
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

인터넷 연결

실습 01과 섹션 01 · 02의 예제는 JSONPlaceholder에 요청합니다.

키 이름은 day20-

같은 주소의 예제끼리는 저장소를 함께 씁니다.
— 45장

01

fetch 응답 처리 심화

상태와 헤더 · 본문은 한 번 · 요청 함수 · 실패의 네 갈래 · 안내 문구 · 시간 제한

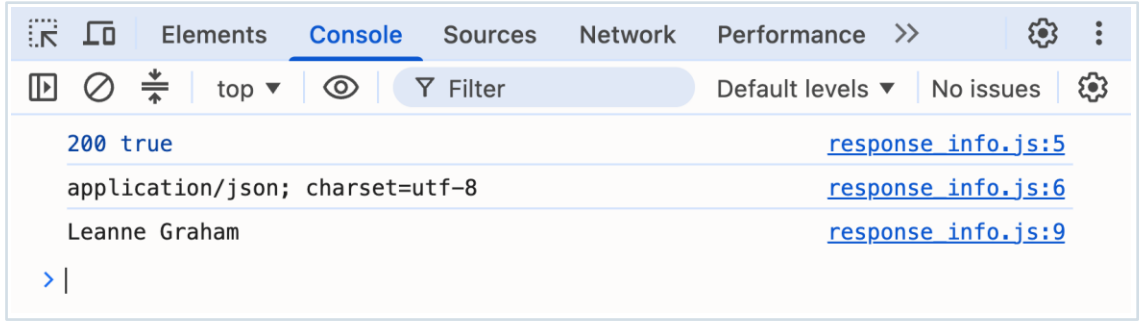
본문을 읽기 전에 보는 상태와 헤더

```
const BASE_URL = "https://jsonplaceholder.typicode.com";

async function look() {
  const response = await fetch(`${BASE_URL}/users/1`);
  console.log(response.status, response.ok);
  console.log(response.headers.get("content-type"));

  const user = await response.json();
  console.log(user.name);
}

look();
```



본문보다 먼저

status · ok — 결과 숫자와 성공 여부
headers.get — 헤더 한 줄
본문은 json()으로 따로 읽음

한 번만 읽을 수 있는 응답 본문

```
const BASE_URL = "https://jsonplaceholder.typicode.com";

async function readTwice() {
  const response = await fetch(`${BASE_URL}/users/1`);
  const user = await response.json();
  console.log(user.name, response.bodyUsed);

  const again = await response.json();
  console.log(again.name);
}

readTwice();
```

한 번 읽으면 끝

bodyUsed — 읽었으면 true
다시 json()하면 TypeError
읽은 값을 변수에 담아 씀

```
Leanne Graham true                                                    body_once.js:6
▶ Uncaught (in promise) TypeError: Failed to execute 'json' on 'Response': body stream already read   body_once.js:8
    at readTwice (body_once.js:8:32)
>
```

Console (위쪽 도구 막대는 잘라 냄)

fetch · ok 확인 · json을 묶은 함수 하나

```
// get_json.js 3~11번 줄
async function getJSON(url) {
  const response = await fetch(url);
  if (!response.ok) {
    const error = new Error(`응답 상태 ${response.status}`);
    error.status = response.status;
    throw error;
  }
  return response.json();
}
```

```
// get_json.js 13~21번 줄 (23번 줄에서 test())
async function test() {
  const post = await getJSON(`${BASE_URL}/posts/1`);
  console.log(post.title);
  try {
    await getJSON(`${BASE_URL}/posts/0`);
  } catch (error) {
    console.log(error.message, error.status);
  }
}
```

```
sunt aut facere repellat provident occaecati excepturi optio reprehenderit   get_json.js:15
▶ GET https://jsonplaceholder.typicode.com/posts/0 404 (Not Found)   get_json.js:4
응답 상태 404 404   get_json.js:19
>
```

오류에 정보를 담아

error.status = 404
받는 쪽이 상태 코드를 봄
빨간 GET 줄은 Chrome이 씀

error의 name과 status로 실패 나누기

실패한 곳	catch가 받는 것	알아보는 법
인터넷 연결 · 서버를 못 찾음	TypeError: Failed to fetch	`error.name === "TypeError"`
응답은 왔지만 404 · 500 …	우리가 던진 Error (응답 상태 404)	`error.status === 404`
본문이 JSON이 아님	SyntaxError (json()이 거부)	`error.name === "SyntaxError"`
정한 시간을 넘김	TimeoutError: signal timed out	`error.name === "TimeoutError"`

catch 하나로 받고, error를 보고 나눕니다

다음 장에서 이 표를 그대로 함수로 옮깁니다.

01 · 안내 문구

실패마다 사람이 읽을 문구로 바꾸기

```
// lib.js 11~15번 줄 — toMessage
function toMessage(error) {
  if (error.name === "TypeError") return "인터넷 연결을 확인해 주세요";
  if (error.status === 404) return "찾는 주소가 없습니다";
  return "잠시 후 다시 시도해 주세요";
}
```

```
// to_message.js 4~11번 줄 — show
async function show(url) {
  try {
    const post = await getJSON(url);
    console.log("성공 :", post.id);
  } catch (error) {
    console.log("실패 :", toMessage(error));
  }
}
```



run — 세 주소를 차례로

posts/1 → 성공 : 1
posts/0 → 찾는 주소가 없습니다
.invalid → 인터넷 연결 확인

기다릴 시간 정하기 — AbortSignal.timeout

```
// timeout.js 3~13번 줄 (1번 줄은 BASE_URL)
async function load(id, ms) {
  try {
    const response = await fetch(`${BASE_URL}/posts/${id}`, {
      signal: AbortSignal.timeout(ms),
    });
    const post = await response.json();
    console.log(`${ms}ms 안에 받음 :`, post.id);
  } catch (error) {
    console.log(`${ms}ms 실패 :`, error.name, error.message);
  }
}
```

```
// timeout.js 15~20번 줄
async function run() {
  await load(1, 5000);
  await load(2, 1);
}

run();
```

```
5000ms 안에 받음 : 1                                    timeout.js:9
1ms 실패 : TimeoutError signal timed out                timeout.js:11
>
```

TimeoutError

error.name이 "TimeoutError"
실습 01은 5초 — 5000

여기까지 정리

1	**상태와 헤더**	status · ok · headers는 본문을 읽기 전에 볼 수 있음
2	**본문**	json()은 한 번만 — 읽은 값은 변수에 담아 씀
3	**getJSON**	fetch · ok 확인 · json()을 함수 하나로, 실패면 status를 담아 던짐
4	**실패의 네 갈래**	TypeError · 상태 코드 · SyntaxError · TimeoutError
5	**시간 제한**	fetch(url, { signal: AbortSignal.timeout(5000) })

다음은 이 함수들을 역할별로 나눠 이어 붙이는 법

02

async/await 구조화

역할별 세 함수 · 오류는 위로 · try에는 요청만 · 차례로와 함께 · 앞의 결과가 필요하면 · allSettled

한 함수에 다 넣던 것을 셋으로 나누기

DAY 18 — loadMembers 하나

- 「불러오는 중…」 쓰기 · 버튼 끄기
- fetch로 요청
- ok가 아니면 throw
- json()으로 읽기
- render — 목록 그리기
- catch — 빨간 안내 문구
- finally — 버튼 다시 켜기

→

통신 `getJSON(url)`

fetch · ok 확인 · json()
실패면 정보를 담아 던짐

화면 `render()`

상태를 보고 그리기만
요청은 하지 않음

흐름 `load()`

불러오는 중 → 요청 → 결과
실패를 받아 상태로

getJSON이 던진 오류를 load의 catch로 받기

```
// propagate.js 3~11번 줄
async function getJSON(url) {
  console.log("getJSON 시작");
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`응답 상태 ${response.status}`);
  }
  console.log("getJSON 끝");
  return response.json();
}
```

```
// propagate.js 13~22번 줄 (24번 줄에서 load())
async function load() {
  console.log("load 시작");
  try {
    await getJSON(`${BASE_URL}/posts/0`);
    console.log("받았습니다");
  } catch (error) {
    console.log("load가 받은 오류 :", error.message);
  }
  console.log("load 끝");
}
```

```
load 시작                                                    propagate.js:14
getJSON 시작                                                 propagate.js:4
▶ GET https://jsonplaceholder.typicode.com/posts/0 404 (Not Found)   propagate.js:5
load가 받은 오류 : 응답 상태 404                               propagate.js:19
load 끝                                                      propagate.js:21
>
```

찍히지 않은 두 줄

「getJSON 끝」·「받았습니다」
throw에서 catch로 바로 넘어감

try에는 실패할 수 있는 요청만 넣기

```
// A — try 안에서 그리기 (8~15번 줄)
async function loadA() {
  try {
    const user = await getJSON(`${BASE_URL}/users/1`);
    render(user);
  } catch (error) {
    console.log("A :", toMessage(error));
  }
}
```

```
// B — try에는 요청만 (17~26번 줄)
async function loadB() {
  let user;
  try {
    user = await getJSON(`${BASE_URL}/users/1`);
  } catch (error) {
    console.log("B :", toMessage(error));
    return;
  }
  render(user);
}
```

```
A : 인터넷 연결을 확인해 주세요                                try_scope.js:13
▶ Uncaught (in promise) TypeError: Cannot read properties of undefined (reading 'city')   try_scope.js:5
    at render (try_scope.js:5:27)
    at loadB (try_scope.js:25:3)
    at async run (try_scope.js:30:3)
>
```

render(4~6번 줄) — user.adress.city : address 오타

서로 상관없는 요청은 함께 기다리기

```
// in_order.js 3~9번 줄 — 차례로
async function inOrder() {
  const start = performance.now();
  const user = await getJSON(`${BASE_URL}/users/2`);
  const posts = await getJSON(`${BASE_URL}/posts?userId=2`);
  const ms = Math.round(performance.now() - start);
  console.log(`차례로 : ${ms}ms`, user.name, posts.length);
}
```

```
// in_order.js 11~20번 줄 — 함께
async function together() {
  const start = performance.now();
  const [user, posts] = await Promise.all([
    getJSON(`${BASE_URL}/users/3`),
    getJSON(`${BASE_URL}/posts?userId=3`),
  ]);
  const ms = Math.round(performance.now() - start);
  console.log(`함께 : ${ms}ms`, user.name, posts.length);
}
```

```
차례로 : 251ms Ervin Howell 10                    in_order.js:8
함께 : 127ms Clementine Bauch 10                  in_order.js:18
>
```

재 보면

차례로 — 두 시간의 합
함께 — 늦은 쪽 하나만큼

앞 요청의 결과를 써야 하면 차례로 기다리기

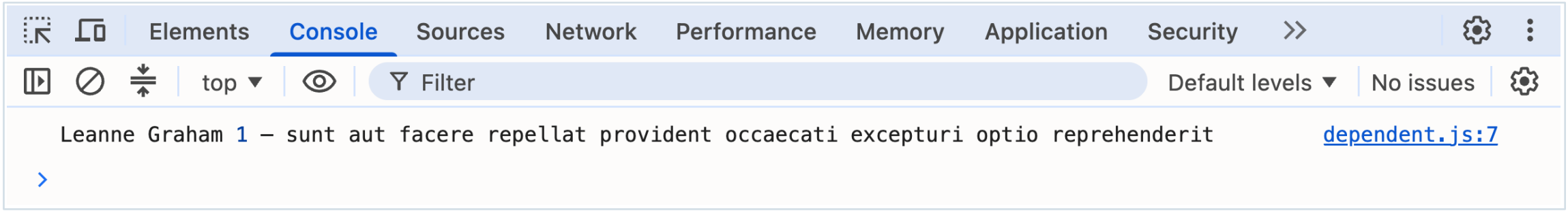
```
const BASE_URL = "https://jsonplaceholder.typicode.com";

async function firstPostOf(username) {
  const users = await getJSON(`${BASE_URL}/users?username=${username}`);
  const user = users[0];
  const posts = await getJSON(`${BASE_URL}/posts?userId=${user.id}`);
  console.log(user.name, user.id, "—", posts[0].title);
}

firstPostOf("Bret");
```

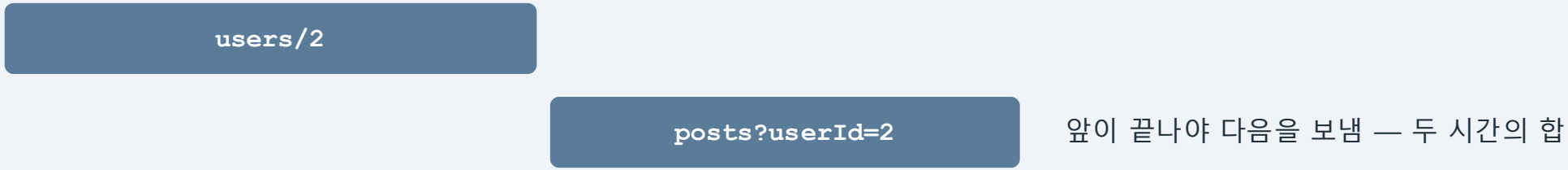
둘째 주소에 첫째 결과

user.id를 알아야 posts를 부름
함께 보낼 수 없음
그래서 await 두 번



차례로 기다리기와 함께 기다리기

차례로 — await 두 번



앞이 끝나야 다음을 보냄 — 두 시간의 합

함께 — Promise.all



둘을 같이 보냄 — 늦은 쪽 하나만큼

막대 길이는 흐름만 나타냅니다 — 실제 시간은 앞 장의 Console

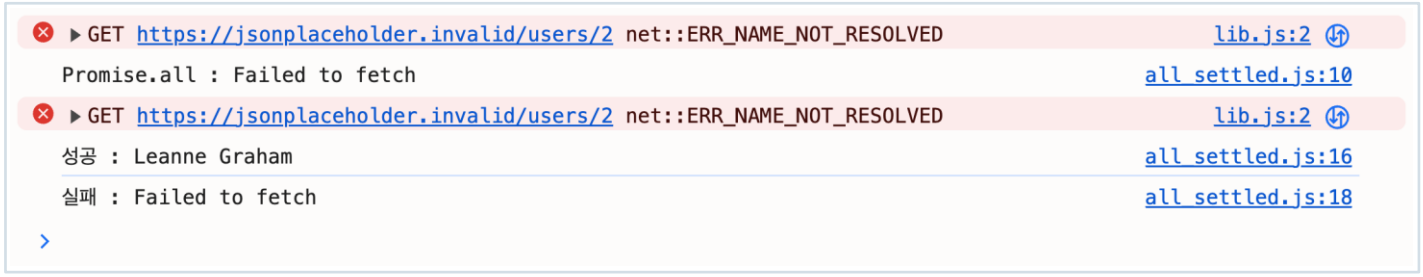
하나가 실패해도 다 받기 — Promise.allSettled

```
// all_settled.js 1~10번 줄
const urls = [
  "https://jsonplaceholder.typicode.com/users/1",
  "https://jsonplaceholder.invalid/users/2",
];

async function run() {
  try {
    await Promise.all(urls.map((url) => getJSON(url)));
  } catch (error) {
    console.log("Promise.all :", error.message);
```

```
// all_settled.js 12~21번 줄

  const results = await Promise.allSettled(urls.map((url) => getJSON(url)));
  results.forEach((result) => {
    if (result.status === "fulfilled") {
      console.log("성공 :", result.value.name);
    } else {
      console.log("실패 :", result.reason.message);
    }
  });
}
```



둘의 차이

all — 하나만 실패해도 전체 실패

allSettled — 하나씩 결과

status · value · reason

여기까지 정리

1	**역할별 세 함수**	통신 getJSON · 화면 render · 흐름 load
2	**오류는 위로**	getJSON이 던진 오류는 await한 load의 catch가 받음
3	**try에는 요청만**	그리기 실수까지 잡으면 안내 문구가 틀림
4	**차례로 · 함께**	앞의 결과가 필요하면 차례로, 아니면 Promise.all
5	**allSettled**	하나가 실패해도 결과를 하나씩 모두 받음

다음은 화면 — 불러오는 중 · 실패 · 성공을 상태로 나눕니다

03

로딩 · 오류 · 성공 상태 분리

흩어진 코드 · 상태 넷 · 상태 객체 · render 하나 · hidden과 disabled · 실습 01

03 · 흩어진 코드

상태 없이 쓴 코드에서 생기는 문제

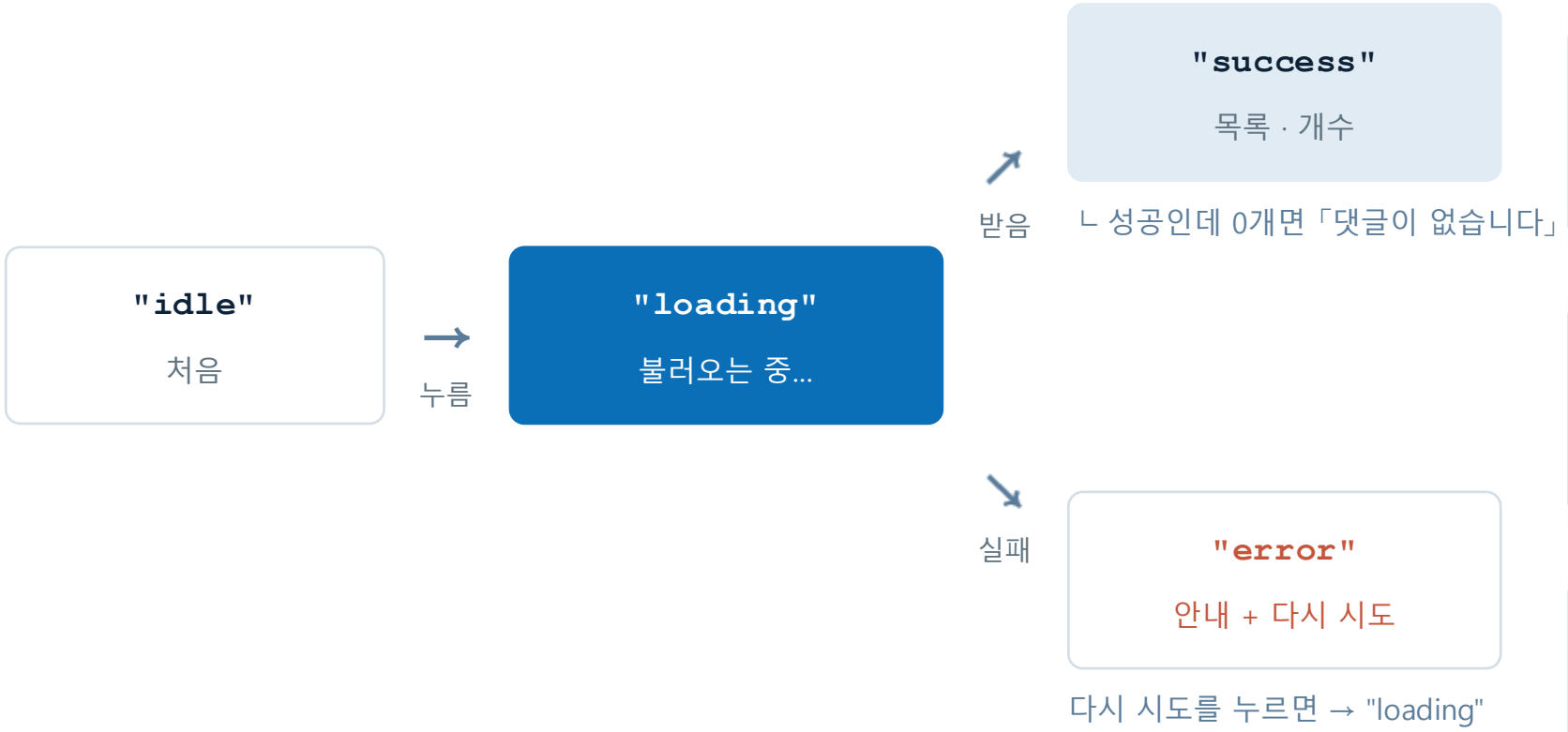
```
// before_state.js 6~20번 줄 — loadComments
async function loadComments() {
  statusEl.textContent = "불러오는 중…";
  try {
    const comments = await getJSON(`${BASE_URL}/posts/${inputEl.value}/comments`);
    statusEl.textContent = `댓글 ${comments.length}개`;
    comments.forEach((comment) => {
      const itemEl = document.createElement("li");
      itemEl.textContent = `${comment.email} — ${comment.name}`;
      listEl.appendChild(itemEl);
    });
  } catch (error) {
    statusEl.classList.add("error");
    statusEl.textContent = toMessage(error);
  }
}
```



네 번째에 누른 뒤 — 15개, 빨간「댓글 5개」

03 · 상태 넷

댓글 보기 화면의 네 가지 상태



상태가 정함

안내 문구 · 목록
버튼 두 개

바뀌는 길

누름 · 받음 · 실패
다시 시도

빈 결과

성공이지만 0개
— 따로 안내

화면에 보일 것을 state 객체 하나에 담기

```
// 3. 상태 — 화면에 보일 것을 객체 하나에
let state = { status: "idle", comments: [], message: "" };

function setState(nextState) {
  state = nextState;
  render();
}
```

status	안내 문구	목록	다시 시도	댓글 보기
"idle"	글 번호를 넣고 …	없음	숨김	켜짐
"loading"	불러오는 중…	없음	숨김	꺼짐
"success"	댓글 5개 · 댓글이 없습니다	있음	숨김	켜짐
"error"	state.message (빨강)	없음	보임	켜짐

상태를 보고 화면을 그리는 함수는 하나

```
// comments.js 39~44번 줄 — 매번 정하는 것
function render() {
  loadBtn.disabled = state.status === "loading";
  retryBtn.hidden = state.status !== "error";
  statusEl.className = state.status === "error" ? "status error" : "status";
  listEl.textContent = "";
```

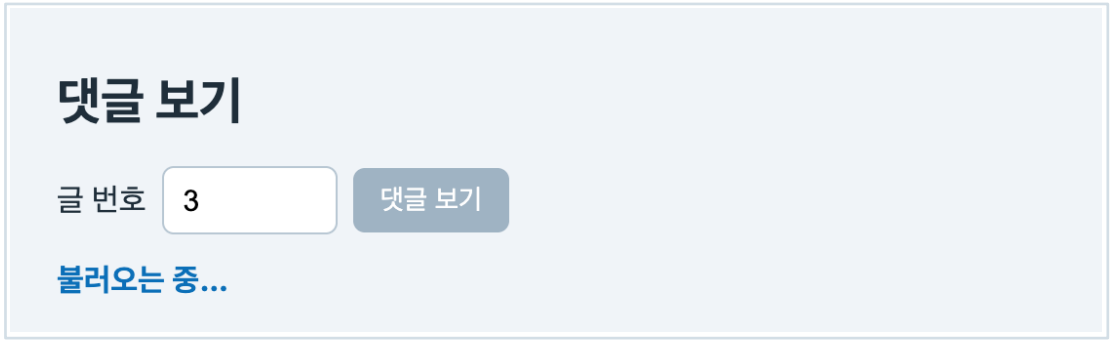
매번 처음부터

버튼 · 다시 시도 · class · 목록을
상태를 보고 다시 정함
— 지울 것을 잊지 않음

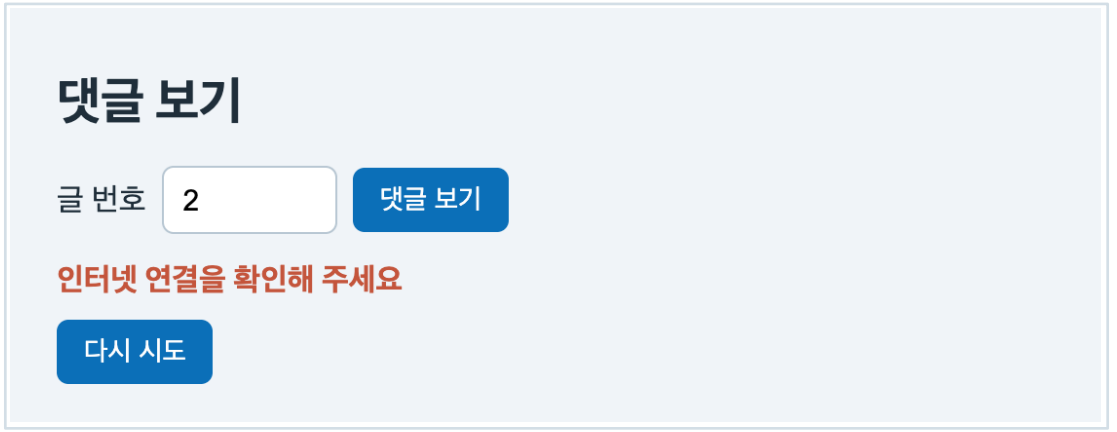
```
// comments.js 45~60번 줄 — 상태마다
  if (state.status === "idle") {
    statusEl.textContent = "글 번호를 넣고 「댓글 보기」를 누르세요";
  } else if (state.status === "loading") {
    statusEl.textContent = "불러오는 중…";
  } else if (state.status === "error") {
    statusEl.textContent = state.message;
  } else if (state.comments.length === 0) {
    statusEl.textContent = "댓글이 없습니다";
  } else {
    statusEl.textContent = `댓글 ${state.comments.length}개`;
    state.comments.forEach((comment) => {
      const itemEl = document.createElement("li");
      itemEl.textContent = `${comment.email} — ${comment.name}`;
      listEl.appendChild(itemEl);
    });
  }
```

상태에 따라 버튼 끄고 숨기기

```
loadBtn.disabled = state.status === "loading";    // 받는 동안 다시 누르지 못하게
retryBtn.hidden = state.status !== "error";       // 실패일 때만 보이게
```



"loading" — 버튼 꺼짐



"error" — 다시 시도 보임

실습 01

댓글 보기

comments.html은 그대로 두고 comments.js를 써서, 글 번호의 댓글을 불러옵니다.
불러오는 중 · 실패 · 성공을 상태로 나누고, 실패하면 다시 시도할 수 있게 합니다.

만들 것

① getJSON — 5초 제한 · ok 확인 · json()
② toMessage — 실패를 안내 문구로
③ state와 setState
④ render — 상태를 보고 그리기
⑤ loadComments — 불러오는 중 → 결과
⑥ submit과 다시 시도에 연결



1번 — 댓글 5개



101번 —「댓글이 없습니다」

실습 01 · 확인

다섯 가지 모습 직접 만들기

상태마다 화면이 맞게 나오는지 하나씩 확인합니다.
불러오는 중과 실패는 개발자 도구 Network의 Throttling으로 만듭니다(DAY 18 33장).

모습	만드는 법	보여야 하는 것
처음	페이지를 연다	글 번호를 넣고 「 댓글 보기」를 누르세요
불러오는 중	Throttling을 Slow 4G · 3G로 두고 누른다	불러오는 중… · 버튼 회색
성공 · 빈 결과	1번으로 누른다 · 101번으로 누른다	댓글 5개 · 댓글이 없습니다
실패	페이지를 연 뒤 Offline으로 두고 누른다	빨간 안내 문구 · 다시 시도
다시 시도	No throttling으로 되돌리고 다시 시도	댓글이 다시 나옴

실습 01

일부러 틀려 보기

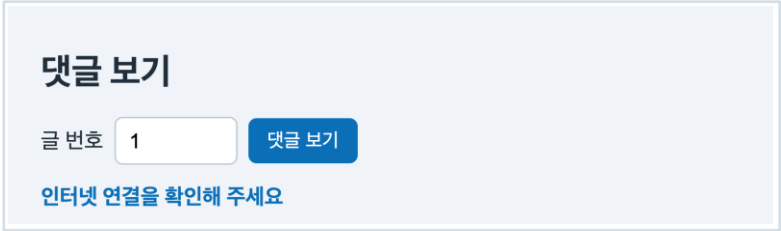
완성한 comments.js를 한 곳씩 바꿔 저장하고, 결과를 본 뒤 되돌립니다.
②와 ③은 Console에 빨간 줄이 뜨고, ④는 화면을 보고 찾습니다.

바꿀 곳	결과
① 69번 줄 주소의 comments를 coments로	「 찾는 주소가 없습니다」와 다시 시도 — 404
② 72번 줄 return; 앞에 // (Offline에서)	파란 글자 안내만 남음 — TypeError (length)
③ 69번 줄의 await 빼기	「 댓글 undefined개」 — TypeError (forEach)
④ 41번 줄 retryBtn.hidden 앞에 // (Offline에서)	빨간 안내만 — 다시 시도 버튼이 안 나옴

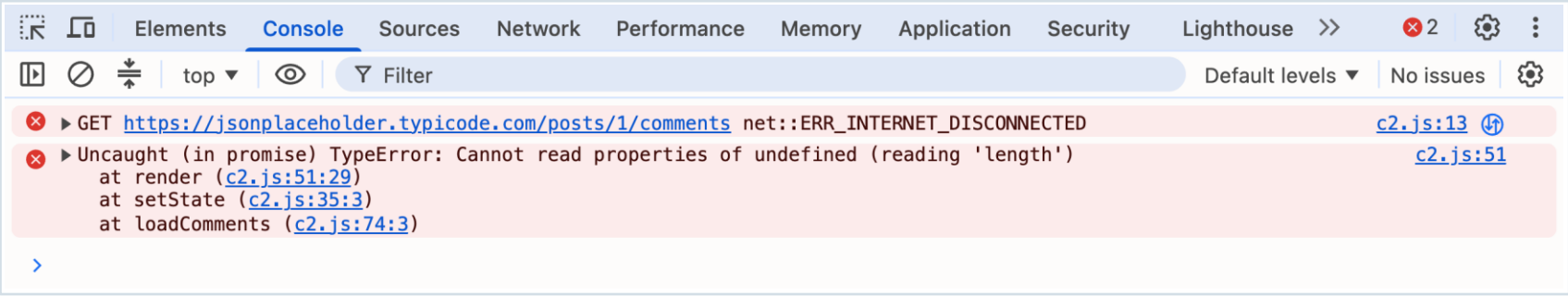
네 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 01 끝

return을 빠뜨리면 — 실패 뒤에도 성공 처리

```
// comments.js 67~74번 줄 (일부러 틀린 곳 ②)
  let comments;
  try {
    comments = await getJSON(`${BASE_URL}/posts/${inputEl.value}/comments`);
  } catch (error) {
    setState({ status: "error", comments: [], message: toMessage(error) });
    // return;
  }
  setState({ status: "success", comments: comments, message: "" });
```



화면 — 파란 안내 문구, 다시 시도 없음



Console — 74번 줄 setState → render 51번 줄에서 멈춤

여기까지 정리

1	**상태**	화면에 보일 것을 정하는 값 — idle · loading · success · error
2	**state 객체**	상태와 받은 데이터 · 안내 문구를 객체 하나에
3	**setState**	새 상태를 넣고 바로 render — 바꾸면 다시 그림
4	**render 하나**	매번 처음부터 — 버튼 · 문구 · 목록을 상태로 정함
5	**빈 결과**	성공이지만 0개 — 실패와 따로 안내

다음은 브라우저 저장소 — 새로고침해도 남는 데이터

04

브라우저 저장소

사라지는 값 · Web Storage · 네 메서드 · 글자만 · JSON · 두 저장소 · 같은 출처 · 실습 02

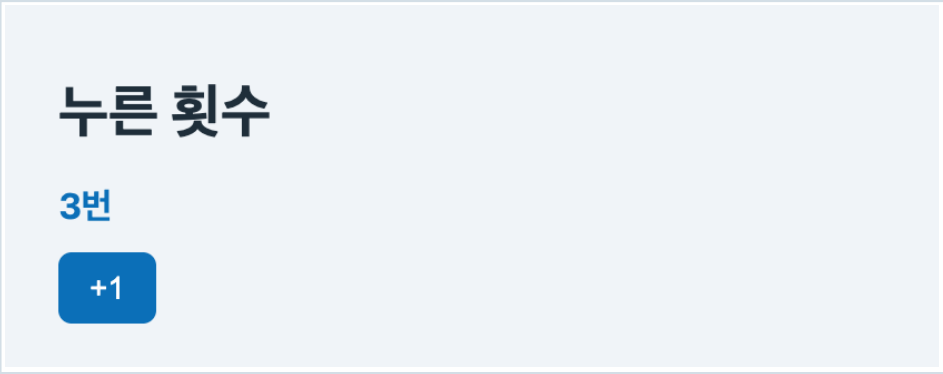
새로고침하면 처음 값으로 돌아가는 변수

```
const countEl = document.querySelector("#count");
let count = 0;

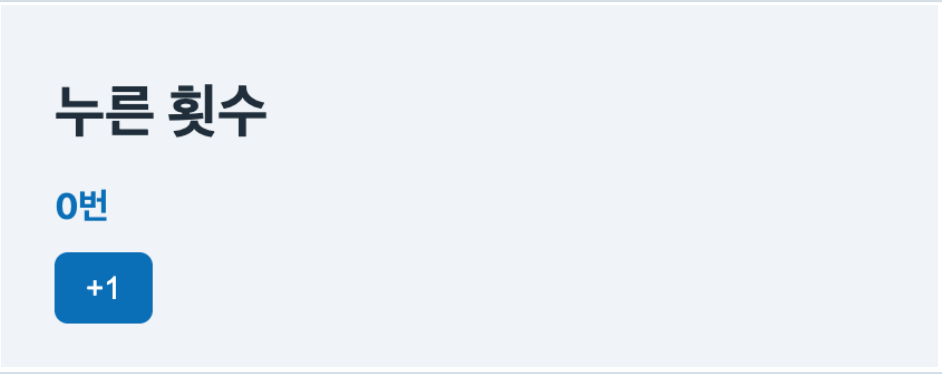
document.querySelector("#add-btn").addEventListener("click", () => {
  count += 1;
  countEl.textContent = `${count}번`;
});
```

페이지와 함께 사라짐

새로고침하면 스크립트가
처음부터 — count는 다시 0
남기려면 브라우저에 저장



+1을 세 번 누른 뒤 — 3번



새로고침한 뒤 — 0번

04 · Web Storage

브라우저에 키와 값을 남기는 저장소

localStorage

Web Storage

출처마다 하나, 브라우저를
닫았다 열어도 남음

sessionStorage

Web Storage

탭마다 따로, 탭을 닫으면
지워짐 (새로고침은 남음)

Cookie

쿠키

요청마다 서버로 함께 감
하나에 보통 4KB까지

IndexedDB

데이터베이스

많은 데이터 · 구조화된
데이터 — 비동기 API

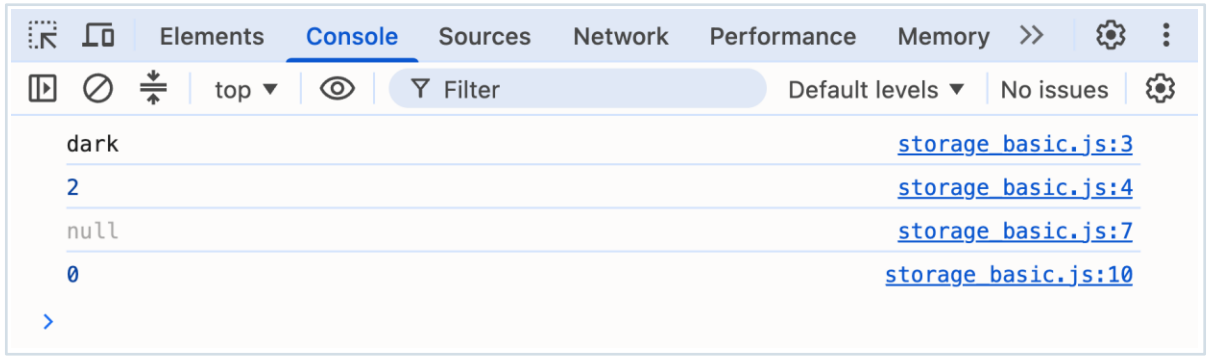
오늘 쓰는 것은 위의 두 칸 — Web Storage

setItem · getItem · removeItem · clear

```
localStorage.setItem("theme", "dark");
localStorage.setItem("lang", "ko");
console.log(localStorage.getItem("theme"));
console.log(localStorage.length);

localStorage.removeItem("lang");
console.log(localStorage.getItem("lang"));

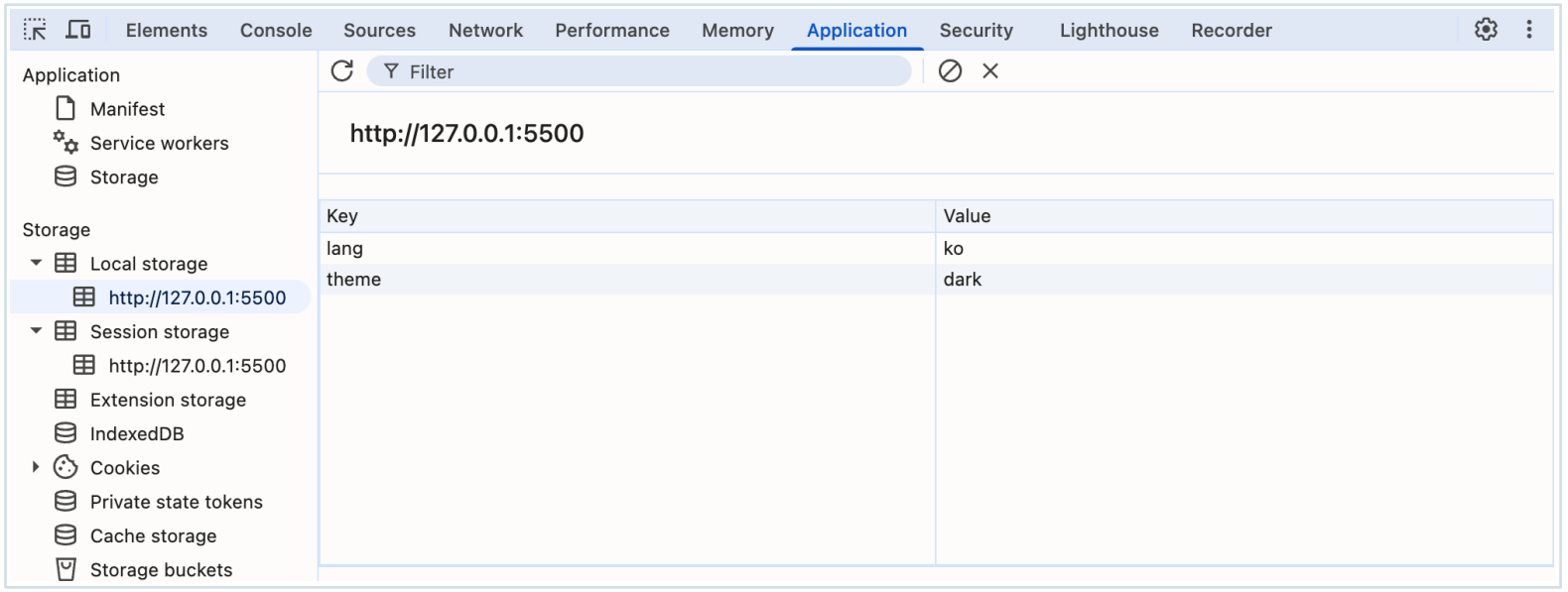
localStorage.clear();
console.log(localStorage.length);
```



네 메서드

setItem(키, 값) — 넣기 · 바꾸기
getItem(키) — 꺼내기, 없으면 null
removeItem(키) · clear()

저장된 값을 보고 고치는 Application 패널



Application > Local storage > http://127.0.0.1:5500

보고 고치기

칸을 두 번 눌러 고침
고친 뒤에는 새로고침

지우기

줄을 골라 Delete
Clear all로 모두

글자만 들어가는 저장소

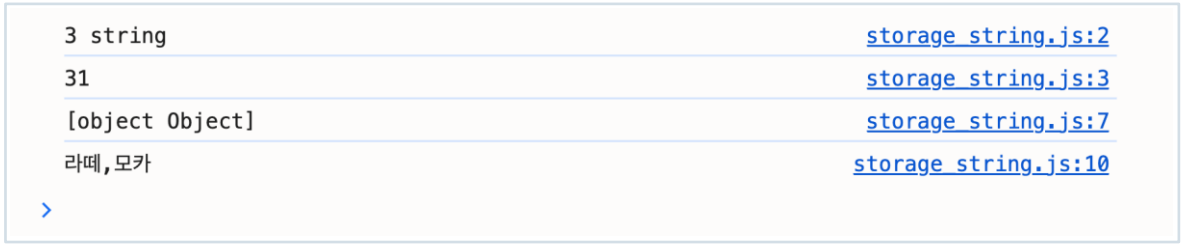
```
localStorage.setItem("count", 3);
console.log(localStorage.getItem("count"), typeof localStorage.getItem("count"));
console.log(localStorage.getItem("count") + 1);

const person = { name: "김오즈" };
localStorage.setItem("user", person);
console.log(localStorage.getItem("user"));

localStorage.setItem("menu", ["라떼", "모카"]);
console.log(localStorage.getItem("menu"));
```

넣는 순간 글자로

3 → "3" — 더하면 "31"
객체 → "[object Object]"
배열 → "라떼,모카"

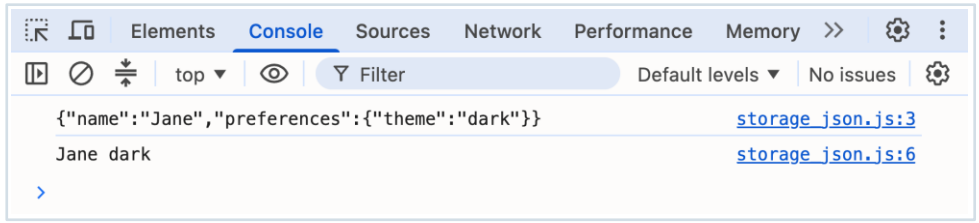


Console (위쪽 도구 막대는 잘라 냄)

배열과 객체는 JSON 글자로 바꿔 넣기

```
const user = { name: "Jane", preferences: { theme: "dark" } };
localStorage.setItem("user", JSON.stringify(user));
console.log(localStorage.getItem("user"));

const storedUser = JSON.parse(localStorage.getItem("user"));
console.log(storedUser.name, storedUser.preferences.theme);
```



넣을 때 · 꺼낼 때

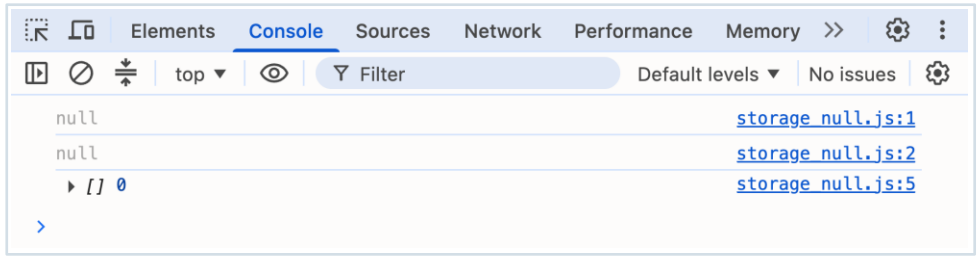
넣을 때 — JSON.stringify
꺼낼 때 — JSON.parse
둘은 늘 짝으로

04 · 처음에는 null

저장한 적이 없으면 null — 빈 배열로 시작

```
console.log(localStorage.getItem("todos"));
console.log(JSON.parse(localStorage.getItem("todos")));

const todos = JSON.parse(localStorage.getItem("todos")) || [];
console.log(todos, todos.length);
```



처음 연 사람

getItem — 없으면 null
JSON.parse(null) — null
|| [] — 빈 배열로 시작

localStorage와 sessionStorage — 남는 기간

	localStorage	sessionStorage
새로고침	남음	남음
같은 주소를 새 탭으로	보임 (함께 씀)	비어 있음 (탭마다 따로)
탭을 닫았다 다시 열기	남음	지워짐
브라우저를 닫았다 다시 열기	남음	지워짐
쓰기 좋은 것	설정 · 할 일 목록	쓰던 글 · 이 탭의 단계

오래 남길 것은 localStorage, 이 탭에서만 쓸 것은 sessionStorage

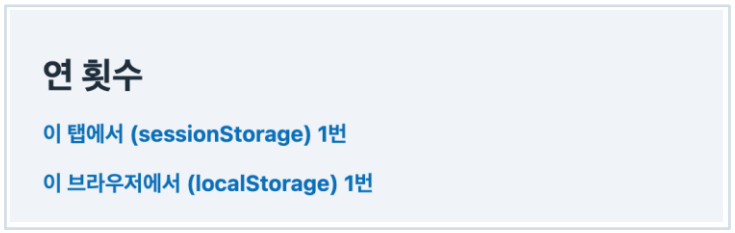
04 · 직접 비교

연 횟수로 보는 두 저장소

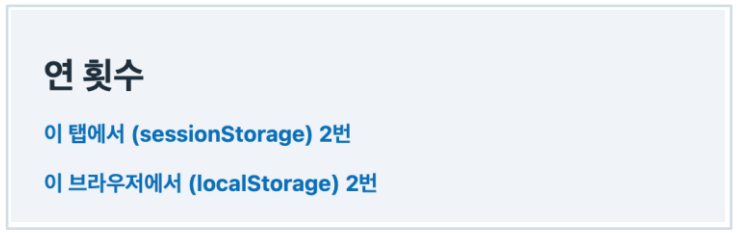
```
const tabCount = Number(sessionStorage.getItem("visits") || 0) + 1;
const allCount = Number(localStorage.getItem("visits") || 0) + 1;

sessionStorage.setItem("visits", tabCount);
localStorage.setItem("visits", allCount);

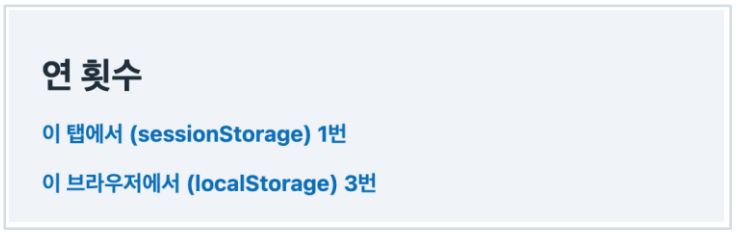
document.querySelector("#tab-count").textContent = tabCount;
document.querySelector("#all-count").textContent = allCount;
```



처음 열기 — 1 · 1



새로고침 — 2 · 2

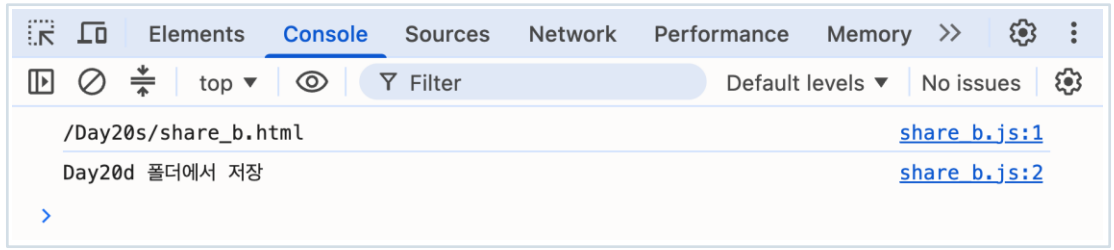


같은 주소를 새 탭으로 — 1 · 3

같은 출처의 페이지가 함께 쓰는 저장소



Day20d/share_a.html — 넣기



Day20s/share_b.html — 다른 폴더에서 읽기

출처 = 프로토콜 + 호스트 + 포트

http://127.0.0.1:5500

Live Server로 연 예제는
모두 이 출처 하나

그래서

키 이름에 날짜

day20-todos · day20-draft처럼
앞에 붙여 부딪치지 않게

넣을 수 있는 양과 넣지 말 것

```
try {
  localStorage.setItem("big", "가".repeat(6 * 1024 * 1024));
} catch (error) {
  console.log(error.name);
  console.log(error.message);
}
console.log(localStorage.getItem("big"));
```

출처마다 5MiB까지

넘으면 QuotaExceededError
비밀번호 · 인증 정보는 넣지 않음
— 같은 출처의 스크립트가 읽음

```
QuotaExceededError                                                    quota.js:4
Failed to execute 'setItem' on 'Storage': Setting the value of 'big' exceeded the quota.   quota.js:5
null                                                                  quota.js:7
>
```

Console (위쪽 도구 막대는 잘라 냄)

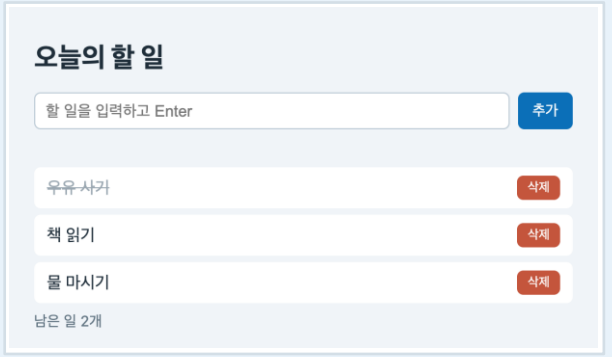
실습 02

새로고침해도 남는 할 일

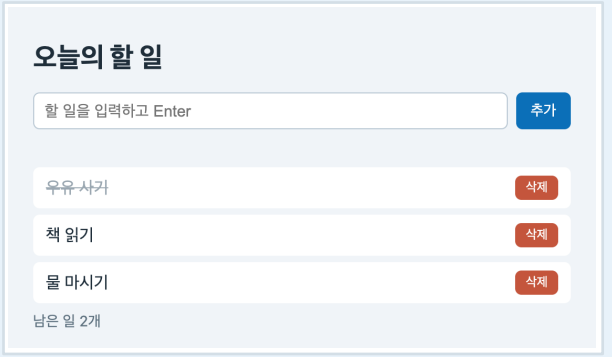
DAY 16의 할 일 목록을 todos 배열 + render로 다시 쓰고, 바꿀 때마다 localStorage에 저장합니다.
입력칸에 쓰던 글은 sessionStorage에 두어, 같은 탭에서 새로고침해도 남게 합니다.

만들 것

① 불러오기 — JSON.parse(…) || []
② save — JSON.stringify로 저장
③ render — 배열을 보고 다시 그리기
④ 추가 · 끝내기 · 지우기 → save → render
⑤ 쓰던 글 — sessionStorage
⑥ submit — 비었으면 안내



셋을 넣고 하나 끝냄



새로고침한 뒤 — 그대로

실습 02

불러오기 · 저장하기 · 바꾸기

```
// todo.js 11~17번 줄
// 1. 불러오기 — 저장한 것이 없으면 null이라 빈 배열로
let todos = JSON.parse(localStorage.getItem(TODOS_KEY)) || [];

// 2. 저장하기 — 배열을 JSON 글자로 바꿔서
function save() {
  localStorage.setItem(TODOS_KEY, JSON.stringify(todos));
}
```

```
// todo.js 50~55번 줄 — 끝내기
function toggleTodo(id) {
  const todo = todos.find((item) => item.id === id);
  todo.done = !todo.done;
  save();
  render();
}
```

열 때 한 번

불러오기

저장된 글자를 parse
없으면 빈 배열

바꿀 때마다

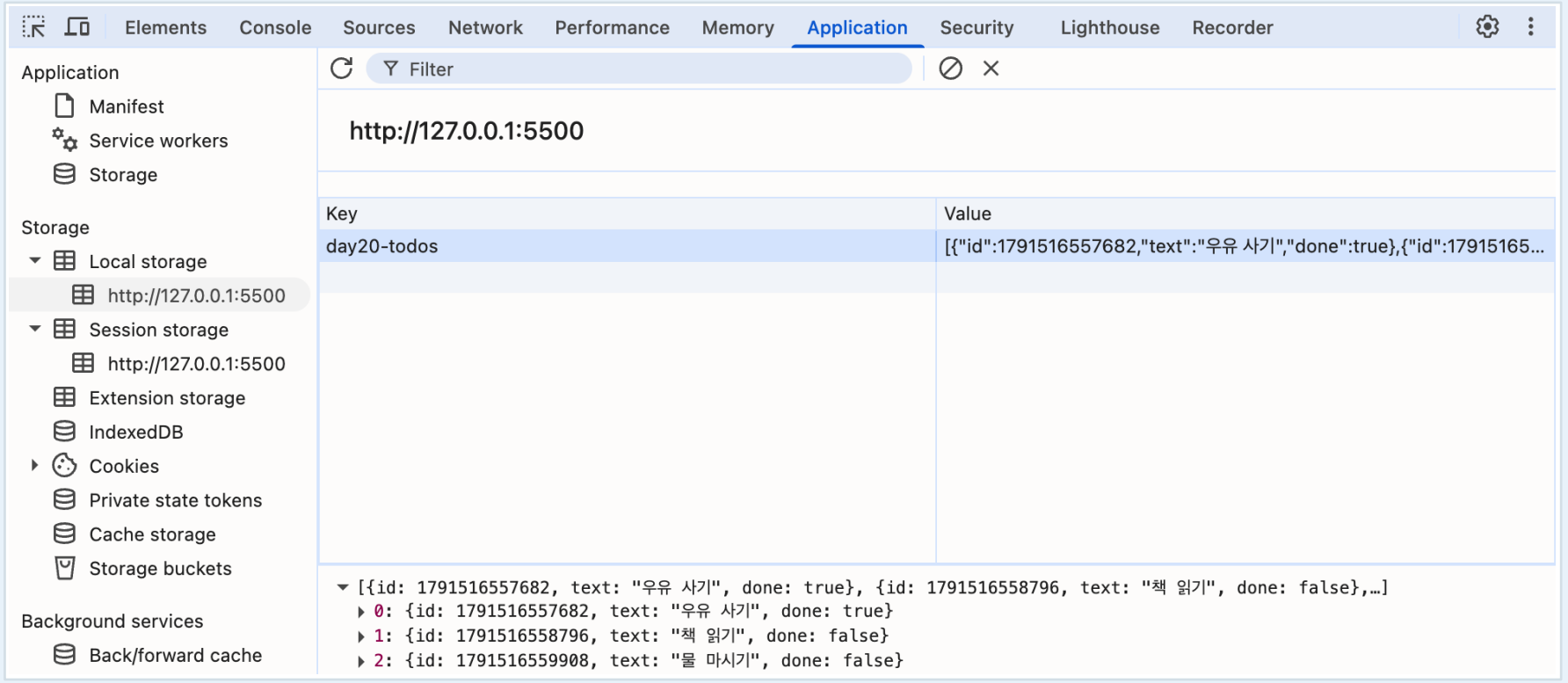
save → render

배열을 stringify해 저장
그리고 화면을 다시

실습 02 · 확인

목록은 Local storage의 day20-todos

Application > Local storage에서 day20-todos 줄을 고르면, 표 아래에 값이 배열로 펼쳐져 보입니다.

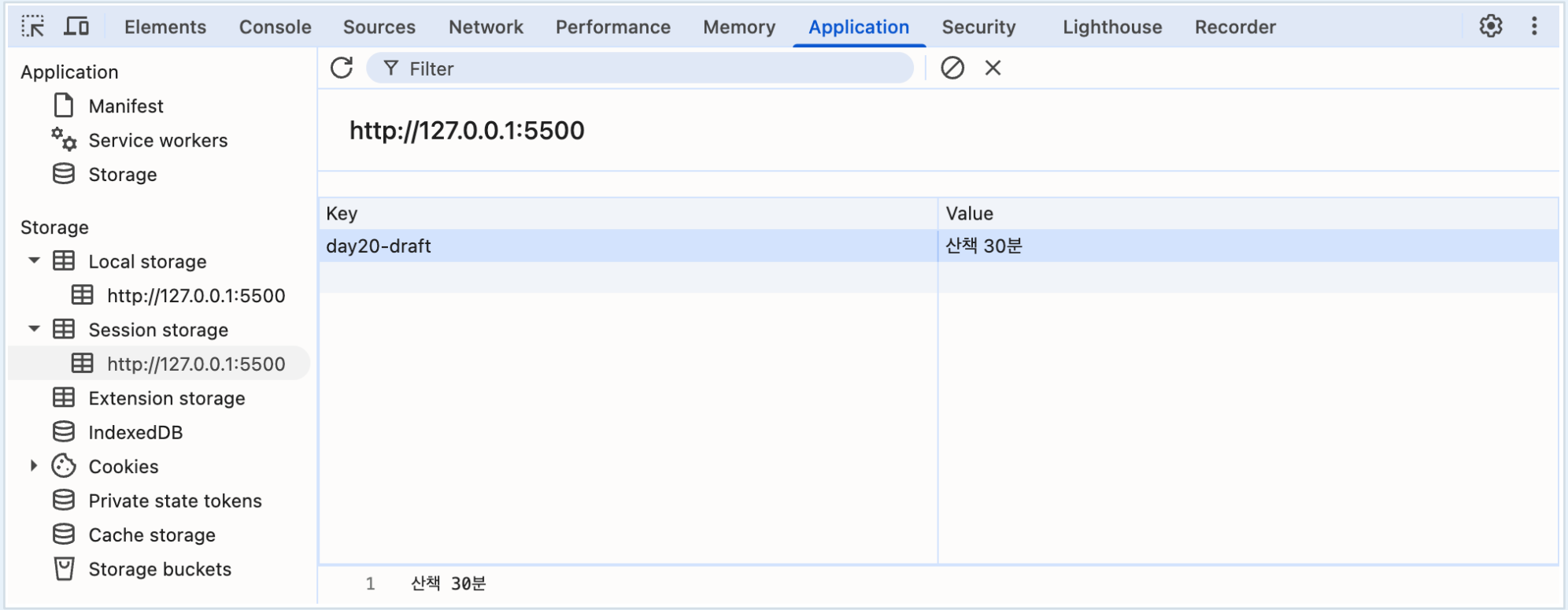


Local storage > http://127.0.0.1:5500 > day20-todos (값 미리보기)

실습 02 · 확인

쓰던 글은 Session storage의 day20-draft

입력칸에 쓰는 동안 input 이벤트마다 sessionStorage.setItem으로 저장합니다.

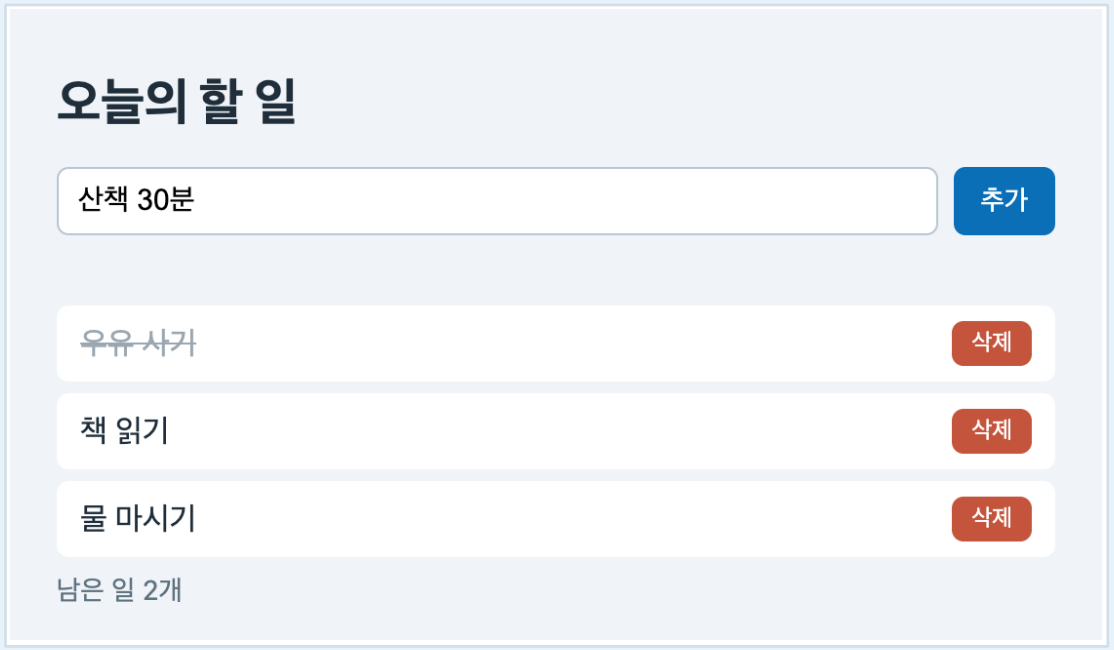


Session storage > http://127.0.0.1:5500 > day20-draft

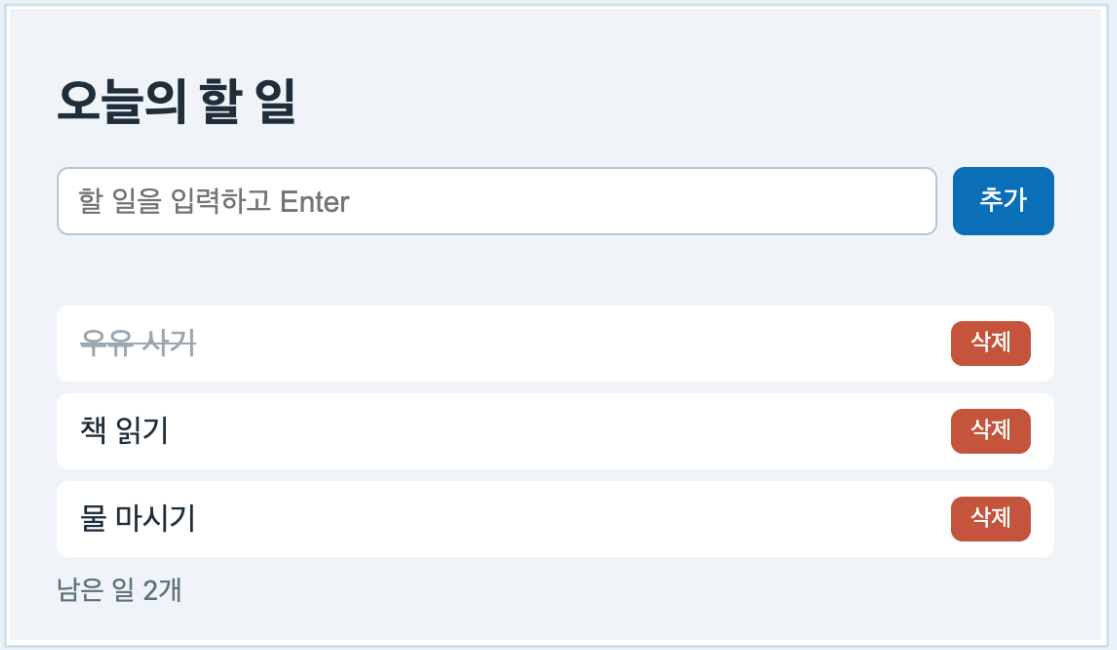
실습 02 · 확인

같은 탭과 새 탭에서 남는 것

「산책 30분」을 쓰다가 새로고침하고, 같은 주소를 새 탭으로도 열어 봅니다.



같은 탭에서 새로고침 — 쓰던 글도 남음



같은 주소를 새 탭으로 — 목록만

실습 02

일부러 틀려 보기

완성한 todo.js를 한 곳씩 바꿔 저장하고, 할 일을 넣고 새로고침해 본 뒤 되돌립니다.
③은 저장소를 비우고(Application 패널의 Clear all) 열어야 보입니다.

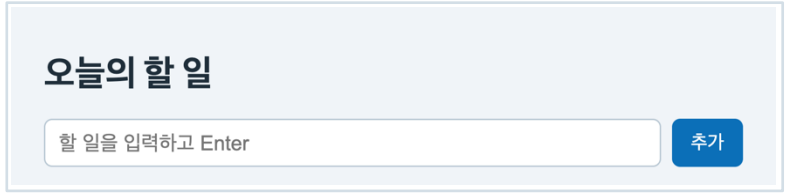
바꿀 곳	결과
① 16번 줄 JSON.stringify(todos)를 todos로	새로고침하면 SyntaxError — 목록이 빔
② 12번 줄의 JSON.parse( ) 빼기	새로고침하면 todos.forEach is not a function
③ 12번 줄의 || [] 빼기 (저장소를 비우고)	처음부터 TypeError — null의 forEach
④ 53번 줄 save(); 앞에 //	새로고침하면 끝낸 표시가 사라짐 — 빨간 줄 없음

네 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

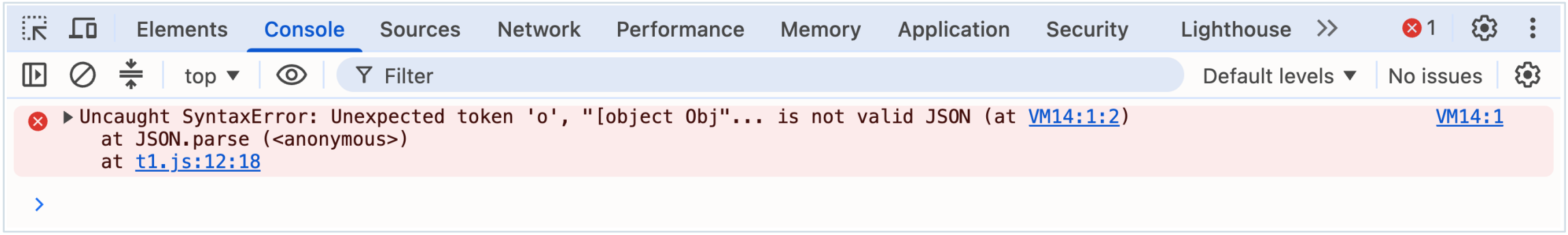
stringify를 빠뜨리면 — 새로고침 뒤 오류

```
// todo.js 11~17번 줄 (일부러 틀린 곳 ①)
// 1. 불러오기 — 저장한 것이 없으면 null이라 빈 배열로
let todos = JSON.parse(localStorage.getItem(TODOS_KEY)) || [];

// 2. 저장하기 — 배열을 JSON 글자로 바꿔서
function save() {
  localStorage.setItem(TODOS_KEY, todos);
}
```



화면 — 목록과 개수가 비어 있음



Console — 저장된 글자 "[object Object]..."를 JSON으로 읽지 못함

여기까지 정리

1	**localStorage**	출처마다 하나 — 브라우저를 닫았다 열어도 남음
2	**sessionStorage**	탭마다 따로 — 새로고침은 남고 탭을 닫으면 지워짐
3	**네 메서드**	setItem · getItem · removeItem · clear
4	**글자만**	넣을 때 JSON.stringify, 꺼낼 때 JSON.parse
5	**처음에는 null**	JSON.parse(…) || [] 로 빈 배열에서 시작

바꿀 때마다 save → render — 새로고침해도 같은 화면

점검

비동기와 저장소에서 자주 나오는 세 가지

!

안내 문구가 엉뚱해요

요청은 됐는데 「인터넷 연결」? try 안에서 그리다 난 실수까지 잡지 않았는지 봅니다.

!

새로고침하면 화면이 비어요

stringify · parse를 짝으로 썼는지, null이면 || []로 시작하는지 봅니다.

!

다른 예제 값이 섞여요

같은 출처는 저장소를 함께 씀. Application 패널에서 키를 보고 이름 앞에 날짜를 붙입니다.

실패와 저장은 '어디서, 무엇이'를 눈으로 확인하면 풀립니다

Console의 error 이름 · Network의 요청 · Application의 키를 차례로 봅니다.

WRAP UP

오늘 정리

개념

상태와 헤더 — status · ok · headers

실패의 네 갈래 — error.name · status

역할별 세 함수 — getJSON · render · load

상태 — idle · loading · success · error

localStorage — 출처마다, 오래 남음

sessionStorage — 탭마다, 탭을 닫으면 지워짐

직접 쓸 수 있어야 하는 것

getJSON으로 요청하고 실패를 문구로

AbortSignal.timeout으로 시간 제한

Promise.all · allSettled로 함께 받기

state + setState + render로 화면 그리기

JSON.stringify · parse로 배열 저장

|| []로 처음 연 사람 맞이하기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

9장. ES2015+ 문법

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

42장. 비동기 프로그래밍
45장. 프로미스
46장. 제너레이터와 async/await

NEXT

DAY 21

모듈 시스템, NPM, Vite

export · import로 파일을 모듈로 나누고,
NPM과 Vite로 프로젝트 환경을 만듭니다.

수고하셨습니다.